

```

/**
 * @module Logging
 * @description Session-aware logging utility using Higher-Order Functions (HOF).
 * Provides a standardized way to log DML, errors, and success states with
 * ANSI color support and environmental filtering.
 */

import type { StatusCode } from "#app/lexicon";
import { SC, StatusMessage } from "#app/lexicon";
import { Session } from "#app/session";
import { ApiError } from "#db/types";

// --- Color Configuration ---
const SYSTEM_NO_COLOR = process.env.NO_COLOR === "true" || process.env.NODE_ENV === "production";

// --- Killbox labels ---
const Red = (l: string) => (SYSTEM_NO_COLOR ? `[${}]:` : `\x1b[31m[${}]\x1b[0m`);
const Green = (l: string) => (SYSTEM_NO_COLOR ? `[${}]:` : `\x1b[32m[${}]\x1b[0m`);
const Yellow = (l: string) => (SYSTEM_NO_COLOR ? `[${}]:` : `\x1b[33m[${}]\x1b[0m`);
const Blue = (l: string) => (SYSTEM_NO_COLOR ? `[${}]:` : `\x1b[34m[${}]\x1b[0m`);
const Magenta = (l: string) => (SYSTEM_NO_COLOR ? `[${}]:` : `\x1b[35m[${}]\x1b[0m`);

// --- Context label only ---
const Cyan = (l: string) => (SYSTEM_NO_COLOR ? `[${}]:` : `\x1b[36m[${}]\x1b[0m`);

// --- Types & Interfaces ---

/**
 * @interface ILogFlags
 * @description Maps to APP_LOGGING JSON in .env to toggle specific log categories.
 */
interface ILogFlags {
  select: boolean;
  update: boolean;
  insert: boolean;
  delete: boolean;
  account: boolean;
  errors: boolean;
  success: boolean;
  [key: string]: boolean;
}

/** @description Shared signature for all Database Manipulation Language (DML) logs. */
type DmlLogger = (context: string, sql: string, cols: string[], args: any[]) => void;
type DmlOps = "select" | "update" | "insert" | "delete";

/**
 * @interface ILogger
 * @description The logging instance passed into the withLog callback.
 */
export type ILogger = {
  success: (context: string, message?: string, code?: StatusCode) => void;
  error: (context: string, message: string, codeOrError: StatusCode | ApiError<any>, err?: any) => void;
  info: (context: string, message: string) => void;
  admin: <T>(context: string, callback: () => T | Promise<T>) => Promise<T | undefined>;
} & Record<DmlOps, DmlLogger>;

const DEFAULT_LOGGING: ILogFlags = {
  select: false,
  update: false,
  insert: false,
  delete: false,
  account: false,
  errors: false,
  success: false,
};

// --- Initialization ---

/** @internal Parses env flags once at startup. */
const getEnvFlags = (): ILogFlags => {
  try {
    if (process.env.APP_LOGGING) {
      return { ...DEFAULT_LOGGING, ...JSON.parse(process.env.APP_LOGGING) };
    }
  } catch (error) {
    console.error(`${Red("Error")} LogConfig: Malformed APP_LOGGING JSON.`, { error });
  }
  return DEFAULT_LOGGING;
};

const FLAGS = getEnvFlags();

// --- Core Functionality ---

/**
 * @function withLog
 * @description HOF that wraps a callback with a session-aware logger.
 * @template T The return type of the provided callback.
 * @param rootContext The name of the initiating function or route being logged.
 * @param context The name of the calling function or route being logged.
 * @param callback The logic to execute, receiving the ILogger instance.
 */
export const withLog = async <T = void>(rootContext: string, callback: (log: ILogger) => Promise<T> | T): Promise<T | any> => {
  const log: ILogger = {
    success: (context, message, code = SC.OK) => {
      if (!FLAGS.success || !(code < 300)) return;
      const { alias } = Session();
      console.log(`${Green("Success")} ${Cyan(context || rootContext)} [${alias}]: ${message || StatusMessage[code]}`);
    },
    error: (context, message, codeOrError, rawError) => {
      if (!FLAGS.errors) return;

      // 1. Resolve the code early
      const incomingCode = codeOrError instanceof ApiError ? codeOrError.code : codeOrError;

      // 2. Silent Guard: If it's a numeric success/redirect code, skip logging as an "Error"
      if (typeof incomingCode === "number" && incomingCode <= 300) {
        return;
      }

      // 3. Normalization (Proceed only if it's a true error)
      const norm = codeOrError instanceof ApiError ? codeOrError.normalize(context, rawError || codeOrError);
      const { alias } = Session();
      console.error(`${Red("Error")} ${Cyan(context || rootContext)} [${alias}]: ${message} (${norm.code}) ${norm.msg}`, norm.data || "");
    },
    info: (context, message) => {
      const { alias } = Session();
      console.log(`${Yellow("Info")} ${Cyan(context || rootContext)} [${alias}]: ${message}`);
    },
  };
};

```

```

...(["select", "update", "insert", "delete"] as const).reduce(
  (acc, key) => {
    acc[key] = (context, sql, cols, args) => {
      if (FLAGS[key]) {
        console.log(`${Blue("SQL")} ${Cyan(context || rootContext)}:`, { sql, cols, args });
      }
    };
    return acc;
  },
  {} as Record<string, DmlLogger>,
),
admin: async <R>(context = "Admin", cb: () => R | Promise<R>): Promise<R | undefined> => {
  const { user, alias, isAdmin } = Session();
  if (!isAdmin) {
    const message = `${Magenta("Security")} ${Cyan(context)} [${alias}/${user}]: Unauthorized Admin Attempt`;
    console.warn(message);
    throw new ApiError(SC.UNAUTHORIZED_ACCESS, message, { ...Session(), context });
  }

  // Audit Trail: Log that an admin action is starting
  if (FLAGS.info) {
    console.info(`${Magenta("Security")} ${Cyan(context)} [${alias}/${user}]: Authorized`);
  }

  return await cb();
},
} as ILogger;

return await callback(log);
};

```